

Optimal Control of a Two-Wheeled Self-Balancing Robot Using LQR and Multi-Objective Genetic Algorithm

Research Project-I Final Report

Submitted

by

Pritam Ranjan Kalita

Masters of Technology in Robotics Engineering

Enrolment No: 24010311358

Under the guidance

of

Dr. Rahul Singhal

Assistant Professor



***Department of Electrical Engineering
School of Advanced Engineering
University of Petroleum and Energy Studies, Dehradun***

CERTIFICATE

This is to certify that, the Research Project-I Report (End-Review) entitled, “**Balancing & Control of Segway Robot Using Classical And Modern Control Techniques**” submitted by **Pritam Ranjan Kalita (590018957)**, in partial fulfillment, for the award of the degree of **Master of Technology in Robotics Engineering** at **Department Electrical Engineering, University of Petroleum and Energy Studies, Dehradun, India**, is a bonafide record of the Project Work carried out by the student under Dr. Rahul Singhal’s supervision and guidance. The content of the report, in full or parts has not been submitted to any other Institute or University for the award of any other degree or diploma.

Date - 18/12/2025

Place - Dehradun

(Signature)

Dr. Rahul Singhal

Assistant Professor

Department of Electrical Engineering

University of Petroleum and Energy Studies

ABSTRACT

This project investigates the balancing and motion control of a two-wheeled self-balancing Segway robot using classical PID control, state-space based Linear Quadratic Regulator (LQR), and a Genetic Algorithm (GA) optimized LQR approach in a MATLAB/Simulink and Simscape Multibody environment. A simplified inverted-pendulum model is first derived using torque and force-based formulations to obtain the transfer function and linear state-space representation required for controller design. A baseline PID controller is implemented and manually tuned to stabilize the chassis tilt angle, followed by an LQR controller whose weighting matrices Q and R are initially selected by engineering judgment. To enhance performance, a GA searches over the elements $[q_1, q_2, q_3, q_4, r]$ within predefined bounds, using Integral Time Absolute Error criteria on position and chassis tilt angle as dual objective functions. Simulation results demonstrate that the GA-tuned LQR controller achieves noticeably reduced rise and settling times in both position and tilt responses compared to the manually tuned LQR controller, indicating the effectiveness of evolutionary optimization for automatic gain selection in self-balancing robotic systems.

ACKNOWLEDGEMENTS

I would like to thank my project supervisor Dr. Rahul Singhal for guiding me through this project. I would also like to extend my heartfelt gratitude to Dr. Devender Kumar Saini for his kind words of encouragement during the difficult phase of the project's development and for sharing his invaluable experience in crafting of the project's academic presentations and reports including this work.

I would also like to express my gratitude to all the authors, researchers, and scholars whose works have been consulted in the development of this work. Finally, I convey my sincere gratitude to my family and my friends Harsh and Jaishree for their unwavering support, understanding, and encouragement throughout this journey.

Pritam Ranjan Kalita
Dehradun, Dec 2025

TABLE OF CONTENTS

1. Introduction	1
1.1 Literature Review	2
1.2 Research Objectives	3
2. Methodology	4
2.1 System Modelling and Assumptions	4
2.2 Dynamic Modelling Approaches	6
2.2.1 Torque-Based Modelling of the Segway Robot	7
2.2.2 Force-Based Modelling of the Segway Robot	10
2.2.3 Comparison of Torque- and Force-Based Approaches ...	14
2.3 Genetic-Algorithm-Based Tuning of LQR Controller	15
3. Results and Discussion	18
3.1 PID Controller Results	18
3.2 LQR Controller Results	19
3.3 GA-Optimised LQR Results	21
3.4 Quantitative Comparison of LQR and GA-LQR	23
4. Conclusion	26
References	27

LIST OF FIGURES

- Figure 1: Model of the self-balancing Segway robot as simulated in MATLAB-Simulink*
- Figure 2: Generalized coordinate definition diagram of the Segway robot*
- Figure 3: Block diagram of PID controller as implemented in Simulink*
- Figure 4: Simulink block diagram of complete PID implementation project*
- Figure 5: State-space / block-diagram representation for force-based LQR model*
- Figure 6: MATLAB output showing selected Q and R matrices (manually tuned LQR)*
- Figure 7: MATLAB output showing LQR gain matrix K*
- Figure 8: Simulink block diagram of LQR controller*
- Figure 9: Simulink block diagram of complete LQR implementation project*
- Figure 10: GA-LQR controller implementation – basic GA-LQR block diagram*
- Figure 11: GA-LQR controller implementation – GA algorithm flowchart / program details*
- Figure 12: Simulink block diagram for extracting ITAE values*
- Figure 13: PID-controlled Segway – chassis tilt and position responses (first PID plot)*
- Figure 14: PID-controlled Segway – additional response plot (second PID figure)*
- Figure 15: Manually tuned LQR – tilt and position responses (first LQR plot)*
- Figure 16: Manually tuned LQR – additional response/control-effort plot*
- Figure 17: GA-optimized LQR – first set of response plots*
- Figure 18: GA-optimized LQR – second set of response plots*
- Figure 19: GA-optimized LQR – third set of response plots*
- Figure 20: GA convergence curve (best fitness versus generation)*
- Figure 21: Pareto front and bar-chart comparison of LQR and GA-LQR performance metrics*

LIST OF TABLES

- Table 1: Summary of key recent contributions on control of two-wheeled self-balancing robots and related inverted-pendulum platforms.*
- Table 2: Model specifications of the Segway robot used in MATLAB-Simulink*
- Table 3: Performance parameters of the Segway using LQR and GA-LQR controllers*

CHAPTER 1

INTRODUCTION

Two-wheeled self-balancing robots (Segway-type robots) are inherently unstable inverted-pendulum systems, and maintaining upright stability requires a fast and reliable closed-loop controller. Classical PID control remains a common baseline because of its simplicity and ease of implementation, and transfer-function-based modeling with Ziegler–Nichol’s tuning has been shown to stabilize a two-wheeled balancing robot in simulation [1]. However, because the plant is nonlinear, underactuated, and strongly coupled, fixed-gain linear controllers can exhibit limited robustness when operating away from the linearization point or under changing conditions.

The Linear Quadratic Regulator (LQR) is among the most popular linear state-feedback controllers used in industrial applications, and it has been successfully implemented for two-wheeled self-balancing robots using linearized state-space models [2-3]. LQR seeks a principled trade-off between state regulation performance and control effort through the weighting matrices Q and R , but practical performance strongly depends on how these weights are selected. To reduce reliance on trial-and-error tuning, LQR weight selection has been optimized using intelligent optimization methods such as Genetic Algorithms (GA) and Bacteria Foraging Optimization Algorithm (BFOA), with reported improvements in balance response and stability for Segway robots [3]. In real implementations, measurement noise from accelero-gyro IMU sensors can degrade LQR feedback quality, and this has motivated Linear Quadratic Gaussian (LQG) designs that combine Kalman filtering with LQR to attenuate sensor noise while preserving optimal regulation [4].

Beyond linear optimal control, researchers have addressed uncertainty, disturbances, and payload changes using robust and adaptive nonlinear methods. [5-7] For example, adaptive-robust control with Lyapunov-based adaptation and fuzzy logic has been applied to Segway balancing under unknown parameters and external disturbances. Robust control for rider-type self-balancing robots has also been enhanced using EKF-based estimation of payload parameters integrated with sliding-mode control to maintain stability across large rider variations. More recently, structural innovation has been explored alongside control design, including a composite two-wheeled self-balancing robot that adds a reaction-wheel

mechanism and employs adaptive (double-fuzzy, anti-integral-saturation) PID to improve static balance, braking, and slope operation in unknown terrain [8].

In this study, a Segway-type two-wheeled self-balancing robot is investigated as a control benchmark, and classical and modern control techniques are implemented and compared on a unified simulation model. A PID controller is designed as a baseline stabilizer following the transfer-function-based modeling approach commonly adopted for two-wheeled balancing robots. An LQR controller is then designed using a linearized state-space model to obtain an optimal state-feedback gain that balances performance and control effort. Finally, a GA-based tuning strategy is used to automate selection of the LQR weighting matrices Q and R , motivated by prior results showing that evolutionary optimization can improve the closed-loop balancing response of LQR-controlled Segway systems.

1.1. Literature review

Table 1. Summary of key recent contributions on control of two-wheeled self-balancing robots and related inverted-pendulum platforms.

Sl. No	Authors	Year	Remarks
[1]	Mudeng et al.	2020	• PID controller design and simulation for two-wheeled balancing robot • Used Ziegler–Nichol’s method for tuning the PID gains.
[2]	Putra et al.	2025	• LQR controller implementation for two-wheeled balancing robot
[3]	Mohammed and Abdulla	2020	• LQR gains optimization using Genetic Algorithm and Bacteria Foraging Optimization Algorithm. • Comparison revealed that the BFOA-LQR controller is faster and a more stable balancing system for the Segway vehicle.
[4]	Feriyonika & Hidayat	2020	• Implemented LQG (Linear Quadratic Gaussian) control for two-wheeled balancing robot. • Combined Kalman filter with Linear Quadratic Regulator (LQR) to reduce

			sensor noise and optimize balance performance.
[5]	Choudhry et al.	2023	• EKF-based Sliding Mode Controller (ESMC) with payload estimation. • Robust control handling unknown mass (up to 200 kg)
[6]	Burkan & Özgüney	2017	• Applied adaptive-robust and fuzzy logic controllers for balancing a two-wheeled Segway robot. • Addresses payload variations and parameter uncertainties.
[7]	Su et al.	2020	• Suggested a nonlinear cascade controller for combined balance and motion control. • Behavior of the robot is estimated using a Kalman filtering algorithm
[8]	Chen et al.	2025	• Self-balancing robot with reaction wheel mechanism. • Used a Adaptive Double-Fuzzy Anti-Integral Saturation PID (ADFAS-PID) for stability.

1.2. Research Objectives

- To study the mathematical model of the Segway self-balancing robot and derive transfer-function and state-space representations suitable for linear controller design (PID and LQR).
- To implement and simulate a PID controller for chassis-tilt stabilization using MATLAB/Simulink.
- To implement and simulate an LQR controller based on the force-input state-space model of the robot.
- To design and implement a Genetic-Algorithm-based procedure for tuning the LQR weighting matrices Q and R , and to compare the performance of the GA-optimized LQR with the manually tuned LQR.

CHAPTER 2

METHODOLOGY

2.1 System Modelling and Assumptions

The physical plant considered in this work is a Segway-type two-wheeled self-balancing robot, modelled as a planar inverted pendulum mounted on a cart that moves along the horizontal axis. The robot consists of two collocated wheels driven by DC motors, a rigid body representing the chassis, and a combined center of mass (COM) located above the wheel axle.

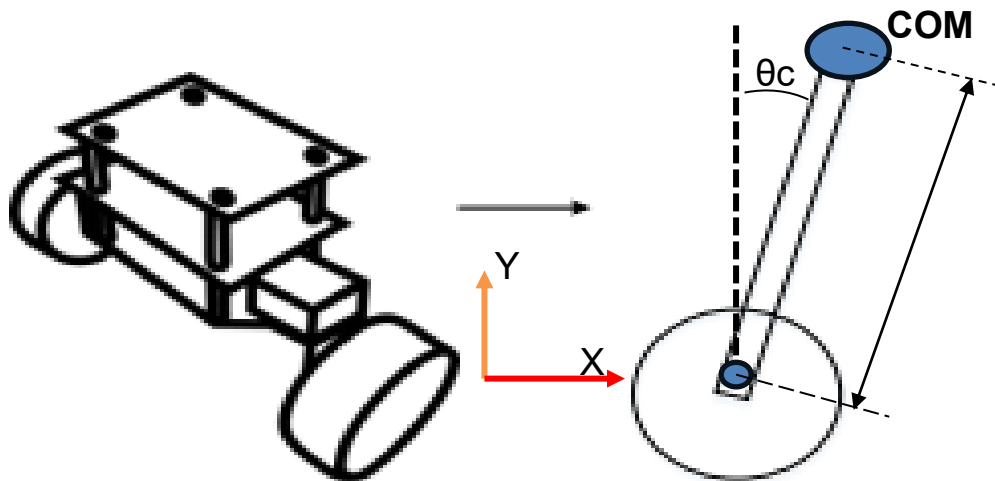


Figure 1. Simplified Model of the Self-Balancing Segway Robot used for Mathematical Analysis

Source: [4] Feriyonika, F. & Hidayat, A. (2020). *Balancing Control of Two-Wheeled Robot by Using Linear Quadratic Gaussian*

For modelling and controller design, the following generalized coordinates are defined:

- x : horizontal displacement of the wheel axle relative to an inertial frame. Rolling without slip is assumed and used to relate wheel rotation to x .
- θ_w : wheel rotation angle measured about the axle, positive in the forward rolling direction.

- θ_c : pitch angle of the chassis measured from the vertical, positive when the robot tilts forward
- τ : net motor torque applied to the drive wheels, used as the control input in the torque-based model.
- F_{input} : equivalent horizontal force at the wheel-ground contact point, representing the control input in the force based model

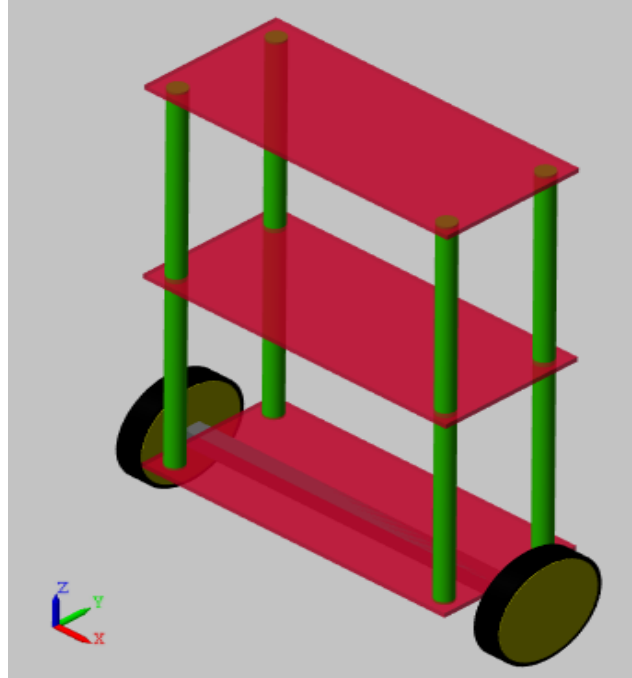


Figure 2. Model of the Self-Balancing Segway Robot as simulated in MATLAB-Simulink

Mass of each plate (kg)	1
Dimensions of each plate (cm)	18 x 8 x 0.3
Mass of each corner support rod (kg)	0.0625
Radius of each corner support rod (mm)	5
Height of each corner support rod (cm)	25
Dimensions of connecting rod between the wheels (cm)	20 x 1 x 1
Mass of connecting rod between the wheels (kg)	0.1
Radius of each wheel (m)	0.0325
Mass of each wheel (kg)	0.05

Table 2. Some Model Specifications of the Segway Robot as simulated in MATLAB-Simulink

From our model specifications, we set some constants which we will use later for defining the robot dynamics & the corresponding Transfer Function and State Space Model equations:

- $\mathbf{b}$ = Viscous Friction Constant = 0.1 N/m/sec
- $\mathbf{M}_c$ = Mass of Chassis = 1 kg
- $\mathbf{M}_w$ = Mass of each Wheel = 0.1 kg
- $\mathbf{L}_c$ = Distance of COM of the chassis from the axis of rotation of wheels = 0.129 m
- $\mathbf{R}_w$ = Radius of each wheel = 0.0325 m
- $\mathbf{I}_c$ = Moment of Inertia of Chassis rotating abt. Wheel axis = $\frac{1}{12} \cdot M_c \cdot L_c^2$
- $\mathbf{I}_w$ = Moment of Inertia of Wheel rotating abt. Wheel axis = $\frac{1}{2} \cdot M_w \cdot R_w^2$

To keep the model analytically tractable and suitable for linear control design, several standard assumptions are made:

- The motion is constrained to a vertical plane; steering dynamics are neglected.
- The wheels are assumed identical, rigid, and rolling without slip on a flat, horizontal surface.
- The chassis is modelled as a rigid body with a fixed centre of mass.
- Motor constraints are neglected. We assume that whatever torque the controller suggests for keeping the robot balanced is available for the motor to supply.
- Aerodynamic drag and higher-order friction effects are neglected; only viscous friction at the wheels is retained where required.

Under these assumptions, the Segway robot can be represented by an inverted-pendulum-on-cart model with two inputs: motor torque and the equivalent horizontal force at the wheel-ground contact point. This unified physical model forms the basis for both the torque-based and force-based dynamic formulations developed in the next subsection.

2.2. Dynamic Modelling Approaches

This subsection presents two equivalent dynamic formulations of the Segway robot: a torque-based model that uses motor torque as the control input, and a force-based model that uses an equivalent horizontal force applied at the wheel-

ground contact point. Both are derived from the same physical system defined in Section 2.1 and will later be used for controller design and comparison.

2.2.1 Torque-Based Modelling of the Segway Robot

In the torque-based formulation, the input to the system is the net motor torque τ applied to the wheels, which generates both wheel rotation and horizontal motion of the cart. The nonlinear equations of motion are obtained by applying Newton–Euler (or Lagrange) equations to the cart–pendulum system, including the wheel–ground interaction through rolling-without-slip constraints.

Non-Linearized Dynamic Model Equations of Segway Robot (Torque Based Approach):

$$\begin{aligned}\tau &= 2 \cdot \ddot{\theta}_w \cdot \left(I_w + M_w \cdot R_w^2 + \frac{M_c \cdot R_w^2}{2} \right) + M_c \cdot L_c \cdot R_w (\ddot{\theta}_c \cos \theta_c - \dot{\theta}_c^2 \sin \theta_c) \\ -\tau &= \ddot{\theta}_c (I_c + M_c \cdot L_c^2) + M_c L_c R_w \ddot{\theta}_w \cos \theta_c - M_c L_c g \sin \theta_c\end{aligned}$$

(Source: Avinash Dubey and Avijit Pandey, “Technical Session on Basics of Self Balancing Robot,” e-Yantra, YouTube video, 2:16:10, Aug. 9, 2022. Available: <https://www.youtube.com/watch?v=nOHky2f58ME>)

The resulting coupled differential equations are expressed in terms of the generalized coordinates θ_c and θ_w , with Torque τ applied to wheels appearing explicitly as the input. These nonlinear equations are then linearized about the upright equilibrium using small-angle approximations for θ_c ($\theta_c \approx 0$).

Linearized Dynamic Model Equations of Segway Robot (Torque Based Approach):

$$\begin{aligned}\tau &= 2 \cdot \ddot{\theta}_w \cdot \left(I_w + M_w \cdot R_w^2 + \frac{M_c \cdot R_w^2}{2} \right) + M_c \cdot L_c \cdot R_w \ddot{\theta}_c \\ -\tau &= \ddot{\theta}_c (I_c + M_c \cdot L_c^2) + M_c L_c R_w \ddot{\theta}_w - M_c \cdot L_c \cdot g \cdot \theta_c\end{aligned}$$

(Source: Adapted from H. Hellman and H. Sunnerman, “Two-Wheeled Self-Balancing Robot: Design and Control Based on the Concept of an Inverted Pendulum,” B.Sc. thesis, KTH Royal Institute of Technology, Stockholm, Sweden, 2015)

The linearized model is written in a form suitable for implementation of a PID controller that directly acts on motor torque. In this project, the torque-based model is used strictly for the design and simulation of the PID controller.

From these Linearized Torque Equations, the equation for Generalized Transfer Function of the robot model is derived assuming that the plant (Segway robot) input is the Torque τ supplied to both the wheels and the plant output is the angle of rotation of the chassis from the vertical θ_c :

$$\frac{\theta_c(s)}{\tau(s)} = \frac{-\frac{k_1 + k_3}{k_5}}{s^2 - \frac{k_2 \cdot k_4}{k_5}}$$

where k_1, k_2, k_3, k_4, k_5 are assumed as follows,

$$\begin{aligned} k_1 &= M_c \cdot L_c^2 + I_c \\ k_2 &= M_c \cdot R_w^2 + 2 \cdot M_w \cdot R_w^2 + 2 \cdot I_w \\ k_3 &= M_c \cdot R_w \cdot L_c \\ k_4 &= M_c \cdot L_c \cdot g \\ k_5 &= k_1 \cdot k_2 - k_3^2 \end{aligned}$$

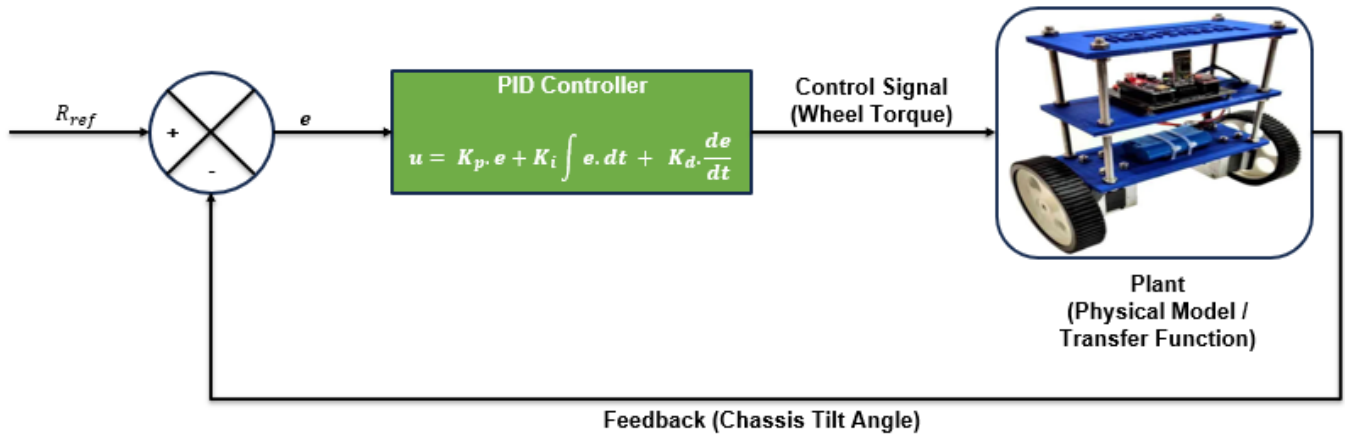


Figure 3. PID Controller Implementation: Basic Block Diagram

Using the **piddtune** method of MATLAB Control System Toolbox the PID gain values for K_p , K_d & K_i are found as follows:

$$K_p = 0.7772$$

$$K_d = 3.7894$$

$$K_i = 0.039853$$

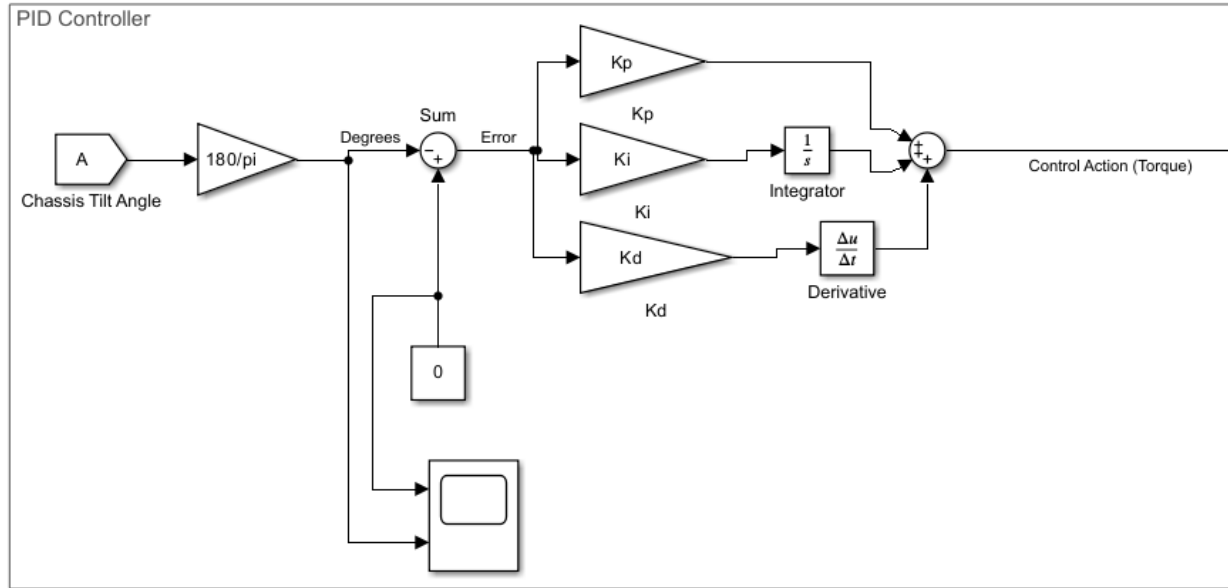


Figure 4. Block Diagram of PID Controller as implemented in Simulink

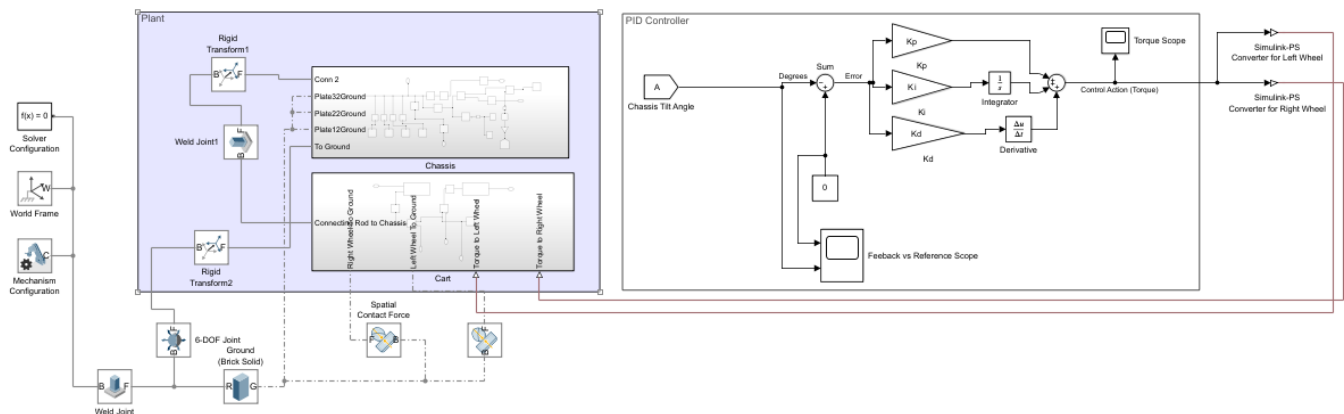


Figure 5: Simulink Block Diagram of Complete PID Implementation Project

2.2.2 Force-Based Modelling of the Segway Robot

In the force-based formulation, the control input is taken as the equivalent horizontal force F applied at the wheel-ground contact point. This is convenient for optimal control design, where the controller is interpreted as commanding a net longitudinal force on the cart, and the mapping to motor torque is handled at the actuator level. Starting from the same free-body diagram, the equations of motion are re-derived with F as the input, leading to an inverted-pendulum-on-cart model expressed in terms of x and θ_c . The nonlinear equations are again linearized around the upright equilibrium and expressed in state-space form with F as the input.

Linearized Dynamic Model Equations of Segway Robot (Force Based Approach):

$$F_{input} = (M_{cart} + M_{chassis}) \cdot \ddot{x} + b \cdot \dot{x} - M_{chassis} \cdot L_c \cdot \ddot{\theta}_c$$

$$(I_{chassis} + M_{chassis} \cdot L_c^2) \cdot \ddot{\theta}_c - M_{chassis} \cdot g \cdot L_c \cdot \theta_c = M_{chassis} \cdot L_c \cdot \ddot{x}$$

(Source: Adapted from H. Hellman and H. Sunnerman, "Two-Wheeled Self-Balancing Robot: Design and Control Based on the Concept of an Inverted Pendulum," B.Sc. thesis, KTH Royal Institute of Technology, Stockholm, Sweden, 2015)

In this project, this force-based linear model is adopted for both the manually tuned LQR controller and the GA-optimized LQR controller. The resulting linear system is written in standard state-space form as:

$$\begin{aligned} \dot{x}(t) &= Ax(t) + Bu(t) & \text{-- State Equation} \\ y(t) &= Cx(t) + Du(t) & \text{-- Output Equation} \end{aligned}$$

where,

$$x(t) = \begin{bmatrix} x \\ \dot{x} \\ \theta_c \\ \dot{\theta}_c \end{bmatrix}$$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & \frac{-(I_{chassis} + M_{chassis} \cdot L_c^2) \cdot b}{k} & \frac{M_{chassis}^2 \cdot g \cdot L_c^2}{k} & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{-M_{chassis} \cdot L_c \cdot b}{k} & \frac{M_{chassis} \cdot g \cdot L_c (M_{cart} + M_{chassis})}{k} & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 \\ \frac{I_{chassis} + M_{chassis} \cdot L_c^2}{k} \\ 0 \\ \frac{M_{chassis} \cdot L_c}{k} \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad D = [0], \quad y(t) = \begin{bmatrix} x \\ \dot{x} \\ \theta_c \\ \dot{\theta}_c \end{bmatrix}$$

$$u(t) = [F_{input}]$$

$$k = I_{chassis}(M_{cart} + M_{chassis}) + M_{cart} \cdot M_{chassis} \cdot L_c^2$$

(Source: Adapted from H. Hellman and H. Sunnerman, "Two-Wheeled Self-Balancing Robot: Design and Control Based on the Concept of an Inverted Pendulum," B.Sc. thesis, KTH Royal Institute of Technology, Stockholm, Sweden, 2015)

In this work, the state vector is chosen as $\mathbf{x} = [x, \dot{x}, \theta_c, \dot{\theta}_c]^T$, and the output vector $\mathbf{y}(t)$ is taken equal to the full state.

For the LQR and GA-LQR designs, the same (A, B) pair is used; only the state-weighting matrix Q and input-weighting scalar R in the quadratic cost

$$J = \int_0^\infty (\mathbf{x}^T Q \mathbf{x} + u^T R u) dt$$

are varied. In the manually tuned LQR, Q and R are selected by engineering judgement, whereas in the GA-LQR case, selected entries of Q and R are treated as decision variables and optimized using the genetic algorithm as described in following section.

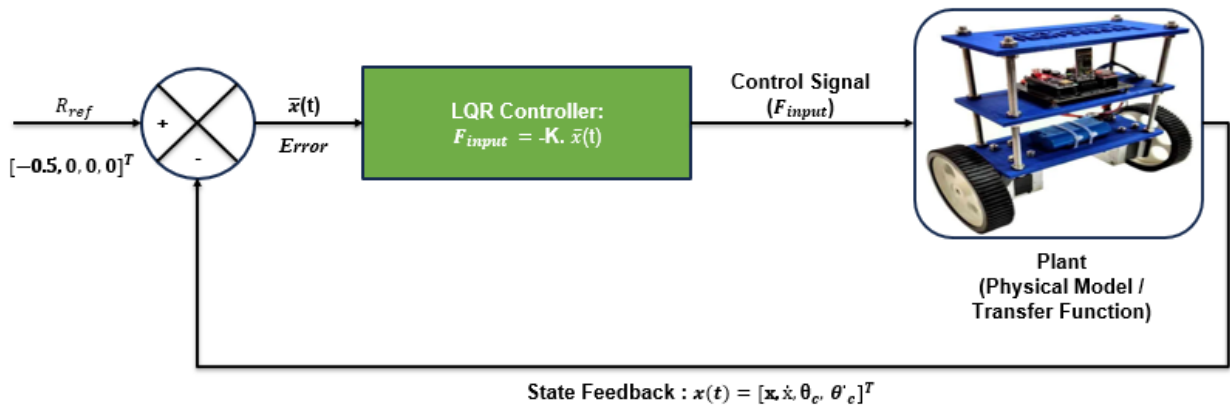


Figure 6. LQR Controller Implementation: Basic Block Diagram

The gain matrix K for the LQR and GA-LQR controllers is calculated in following steps:

1. Given A and B (from Plant).
2. Choose Q and R.
3. Solve Algebraic-Riccati Equation for S (yields multiple solutions for S)

$$A^T S + SA - SBR^{-1}B^T S + Q = 0$$
4. Compute $K = R^{-1}.B.S$ (yields multiple solutions for K)
5. Calculate the $eig(A - BK)$ for all the Ks.
6. Choose K that yields the most stable system (most negative eigen values).

MATLAB provides a function **lqr (A, B, Q, R)** that performs all the above-mentioned steps and provides us with a final K value.

For the manually tuned LQR Controller, the following values of Q and R matrices are used:

$$Q = \begin{bmatrix} q1 & 0 & 0 & 0 \\ 0 & q2 & 0 & 0 \\ 0 & 0 & q3 & 0 \\ 0 & 0 & 0 & q4 \end{bmatrix} = \begin{bmatrix} 100 & 0 & 0 & 0 \\ 0 & 10 & 0 & 0 \\ 0 & 0 & 100 & 0 \\ 0 & 0 & 0 & 10 \end{bmatrix}$$

$$R = [r] = [0.1]$$

Together with the A and B matrices as per the system's model specifications, they produce the following value of K matrix:

K =

-31.6228 -26.0662 100.7381 15.3156

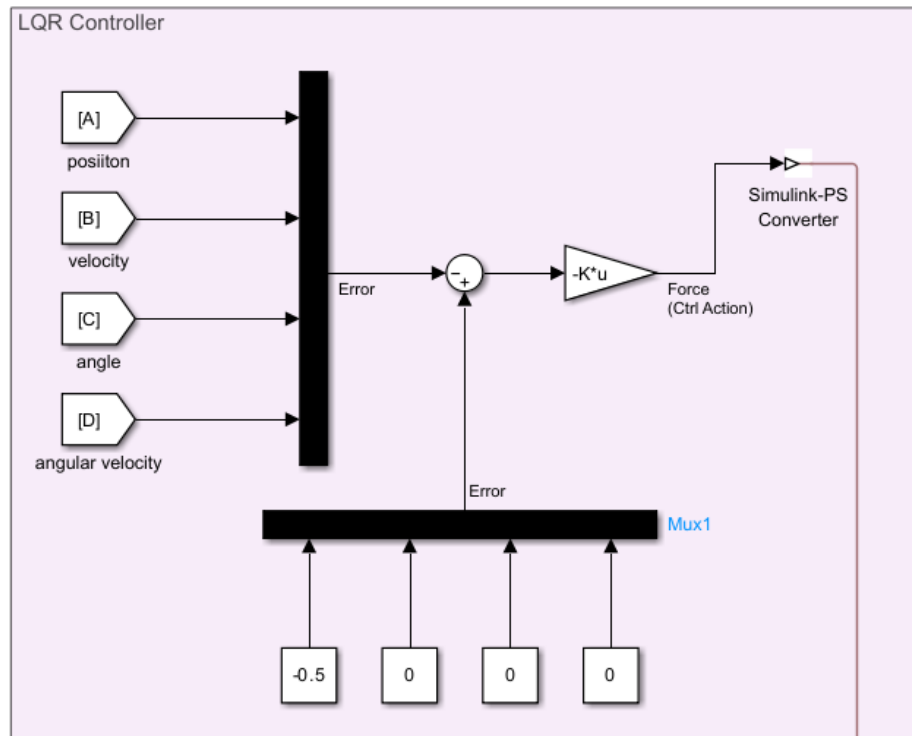


Figure 7. LQR Controller Implementation: Simulink Block Diagram

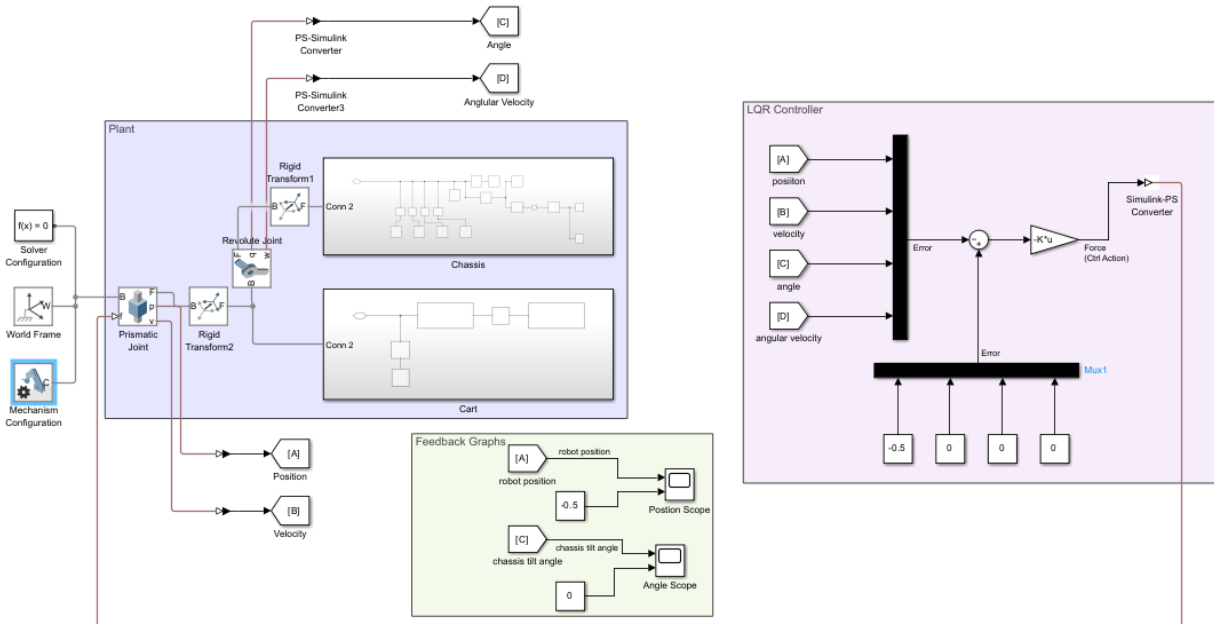


Figure 8. LQR Controller Implementation: Simulink Block Diagram of Complete Project

2.2.3. Comparison of Torque-based and Force-based Approaches

In this project, two actuator representations are considered for the same Segway dynamics: a torque-based input and a force-based input. The **torque approach** treats the controller output as the net motor torque τ_{motor} that must be applied at the wheels to keep the robot balanced. This view is physically intuitive at the actuator level and is therefore adopted for the PID controller implementation. However, simulating the full torque-driven dynamics requires handling the wheel rotational states explicitly and is computationally more demanding.

The **force approach** instead assumes that the controller outputs a horizontal push force F which, if applied at the wheel–ground contact point, would keep the cart–pendulum system balanced. This commanded force is then mapped to the corresponding wheel torque using the relation

$$\tau_{motor} = \frac{F \cdot r_{wheel}}{2}$$

(Source: Adapted from H. Hellman and H. Sunnerman, “Two-Wheeled Self-Balancing Robot: Design and Control Based on the Concept of an Inverted Pendulum,” B.Sc. thesis, KTH Royal Institute of Technology, Stockholm, Sweden, 2015)

where r_{wheel} is the wheel radius and the factor of two accounts for the pair of drive wheels. This formulation is widely used in textbooks and research papers on inverted-pendulum and self-balancing robots and fits naturally into the standard state-space framework employed for LQR design. Consequently, the force-based model is used for both the manually tuned LQR controller and the GA-optimized LQR controller in this work.

Both approaches are dynamically equivalent near the operating point; the choice between them is guided by convenience at the controller-design level rather than by any fundamental difference in the underlying physics. The torque-based model is preferred when designing and implementing a classical PID that directly drives motor torque, while the force-based model is preferred for optimal state-feedback designs where the control variable is a generalized input force acting on the cart.

2.3 Genetic-Algorithm–Based Tuning of LQR Controller

The conventional LQR controller in Section 2.2 uses hand-selected weighting matrices Q and R . To obtain a better trade-off between position regulation, chassis tilt suppression, and control effort, a Genetic Algorithm (GA) is used to tune these weights automatically using the force-based state-space model of the Segway robot.

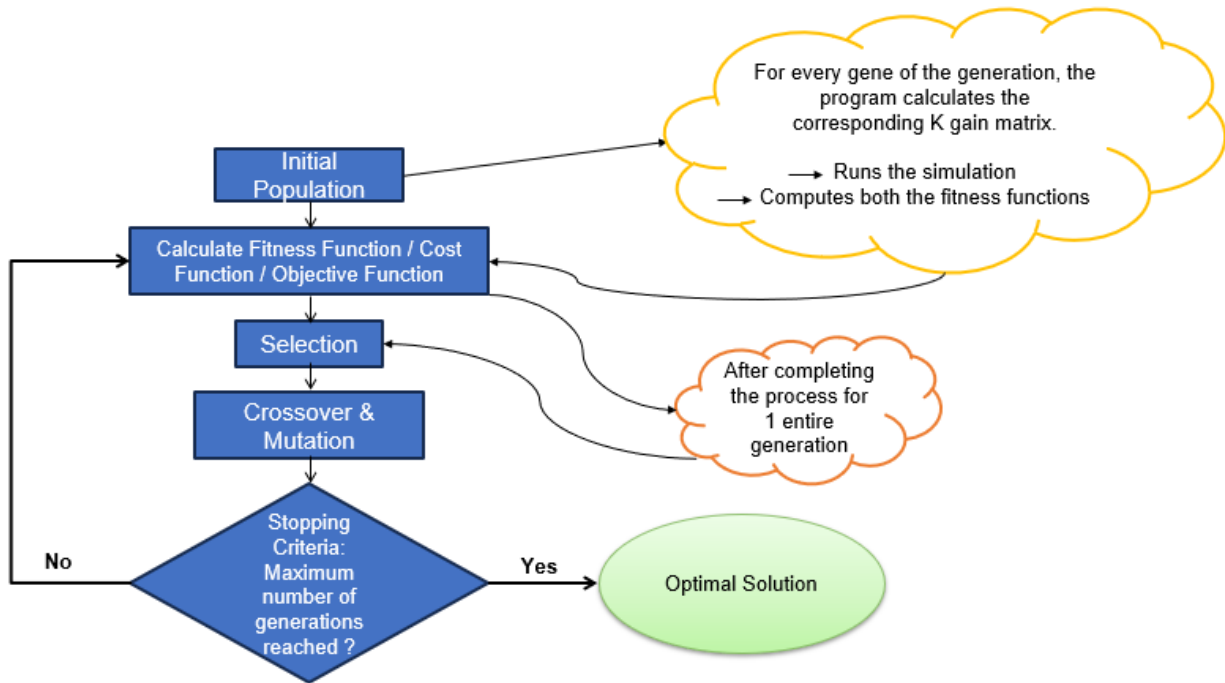


Figure 9. Genetic Algorithm Implementation for Selection of Optimal Q & R Values for LQR Controller

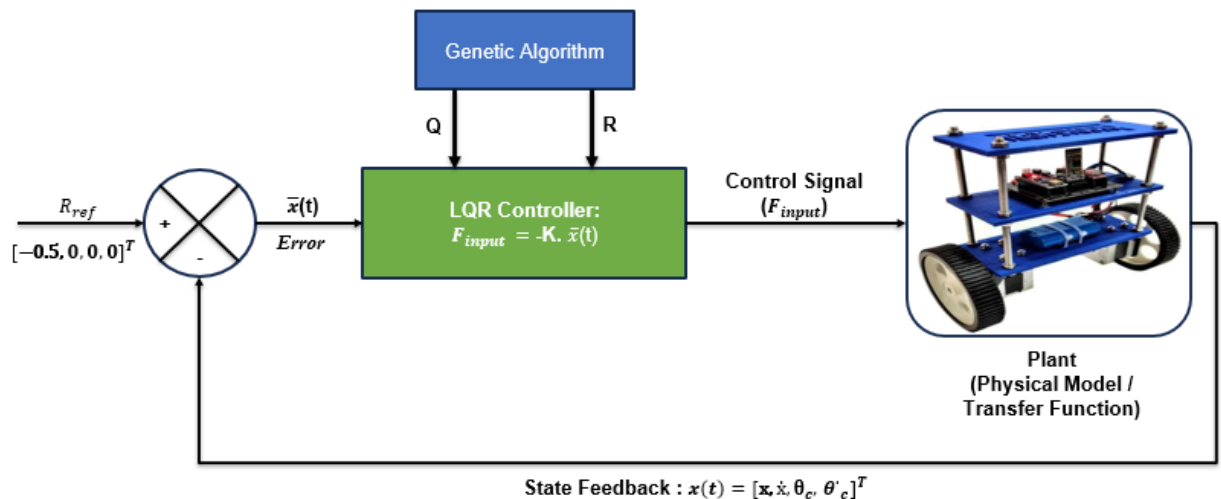


Figure 10. Genetic Algorithm Implementation for Selection of Optimal Q & R Values for LQR Controller

In this work, the GA optimizes five decision variables collected in a single gene:

$$[q_1, q_2, q_3, q_4, r]$$

where the LQR weight matrices are chosen as:

$$Q = \text{diag}(q_1, q_2, q_3, q_4), \quad R = [r]$$

Each candidate vector $[q_1, q_2, q_3, q_4, r]$ therefore defines one complete LQR design.

The variables are constrained within prescribed bounds to keep the closed-loop system stabilizable and to avoid unrealistically large forces:

- Lower bounds: $[1, 1, 1, 1, 0.001]$
- Upper bounds: $[10^6, 10^5, 10^5, 10^4, 10]$

Any individual outside these ranges is rejected or heavily penalized.

The objective of the GA is to minimize the Integral Time Absolute Error (ITAE) of the closed-loop responses. Two ITAE terms are considered, one for cart position error and one for chassis tilt angle error, evaluated over a fixed simulation horizon T :

$$J_{\text{ITAE, pos}} = \int_0^T t |e_x(t)| dt$$

$$J_{\text{ITAE, tilt}} = \int_0^T t |e_{\theta_c}(t)| dt$$

These two objectives are combined into a single scalar fitness (for example as a weighted sum), so that lower ITAE values correspond to better time-domain performance with faster settling and smaller overshoot.

For each candidate gene, the corresponding Q and R are passed to MATLAB's **lqr** (**A**, **B**, **Q**, **R**) routine to compute the feedback gain K . The GA then runs the nonlinear Simulink model of the Segway robot with this K for a standard test (initial tilt disturbance), and the resulting position and tilt responses are used to compute the ITAE-based fitness. Unstable responses or simulations that violate predefined limits on F are assigned a large penalty to discourage such solutions.

The GA is configured with a population size of 40 individuals and is run for 40 generations, so that each generation carries 40 candidate genes through selection, crossover, and mutation operators. Standard tournament selection, single-point crossover, and random mutation are employed to explore the search space while retaining the best individuals found so far (elitism). At the end of 40 generations, the gene with the minimum fitness value is selected as the final GA-optimized design, and its corresponding Q , R , and K are used in the GA-LQR controller. The performance of this optimized controller is later compared against the manually tuned LQR in terms of rise time and settling time for both position and chassis tilt angle.

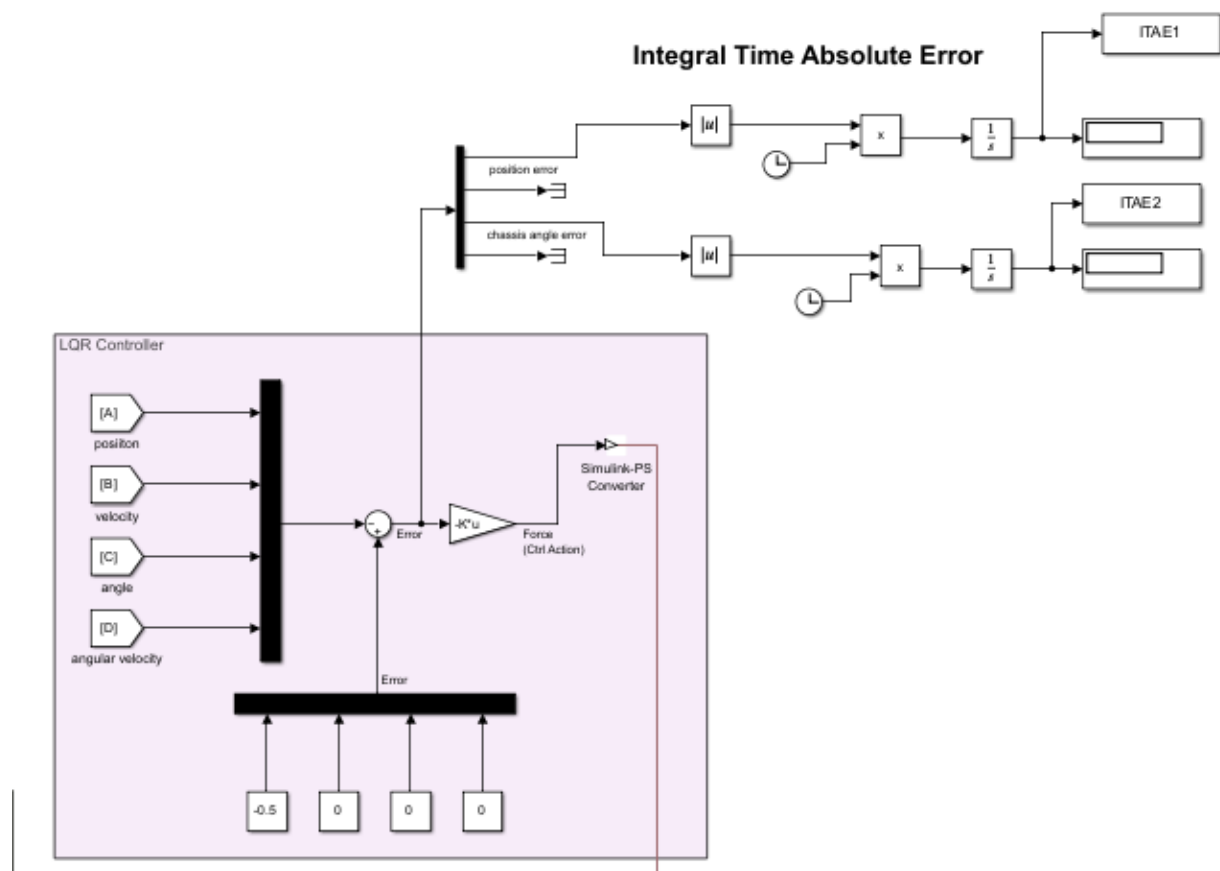


Figure 10. Simulink Block Diagram for Extracting the Integral Time Absolute Error Values

The detailed comparison of manually tuned LQR and GA-LQR controllers, based on the resulting rise and settling times, is presented in Chapter 3 (Results and Discussion).

CHAPTER 3

RESULTS AND DISCUSSION

This chapter presents and compares the simulation results obtained with the PID, LQR, and GA-optimized LQR controllers on the Segway robot model. The focus is on chassis tilt stabilization and cart position regulation under identical initial conditions.

3.1 PID Controller Results

The PID controller designed on the torque-based transfer function is first evaluated to verify basic stabilisation capability. The closed-loop response shows that the chassis tilt angle returns to the upright position without divergence, confirming that the torque-based model and PID gains are correctly implemented.

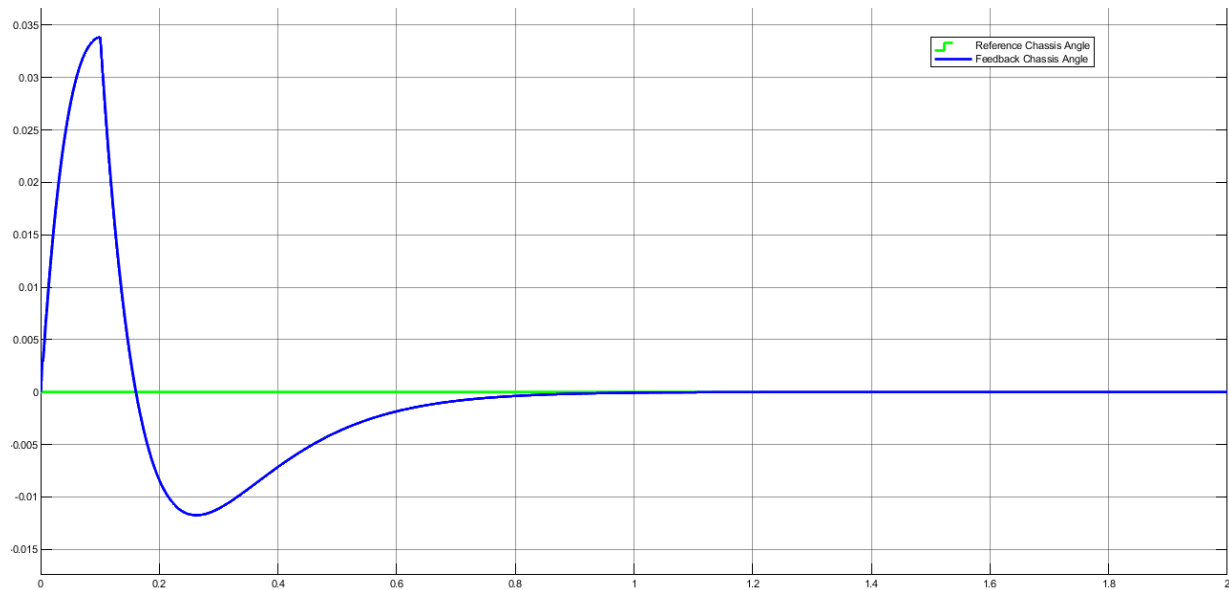


Figure 11: Chassis Tilt Angle (θ) v/s Time Graph from PID Controlled Segway Robot Simulation in MATLAB-SIMULINK

However, the response exhibits noticeable oscillations and relatively long settling time, especially in the cart position. Since the PID controller is implemented as a SISO loop using only the chassis tilt angle as feedback, it primarily focuses on keeping the robot upright and does not explicitly regulate the cart position. As a

result, the robot may stabilize in an offset location, and the controller does not fully exploit the available state information. This motivates the use of multivariable optimal state-feedback in the following sections.

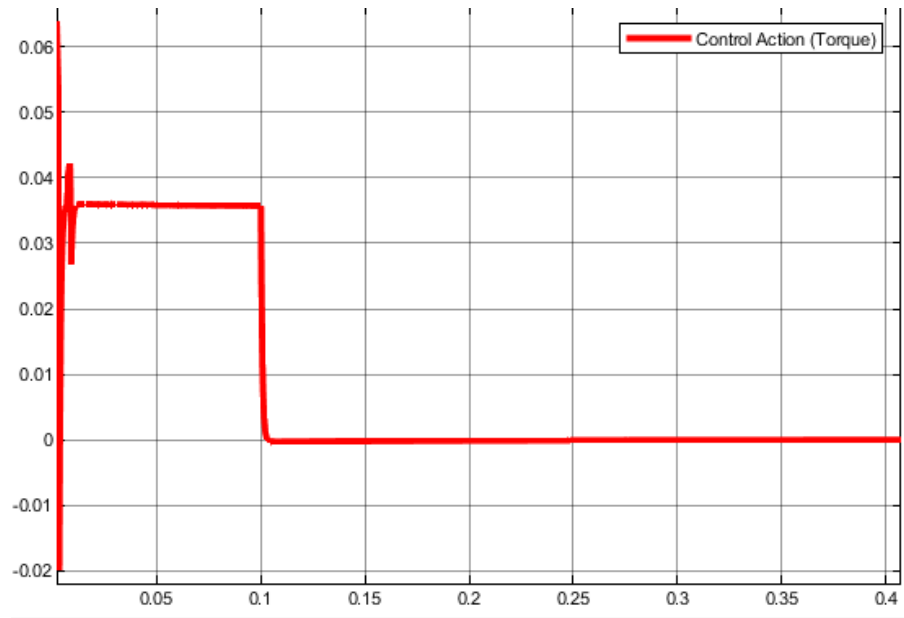


Figure 12: Control Action/Torque (τ) v/s Time Graph from PID Controlled Segway Robot Simulation in MATLAB-SIMULINK

3.2 LQR Controller Results

Using the force-based state-space model and manually tuned Q, R , the LQR controller significantly improves tilt stabilisation compared with PID. The tilt angle converges to zero with smaller oscillations, and the cart position settles with reduced overshoot.

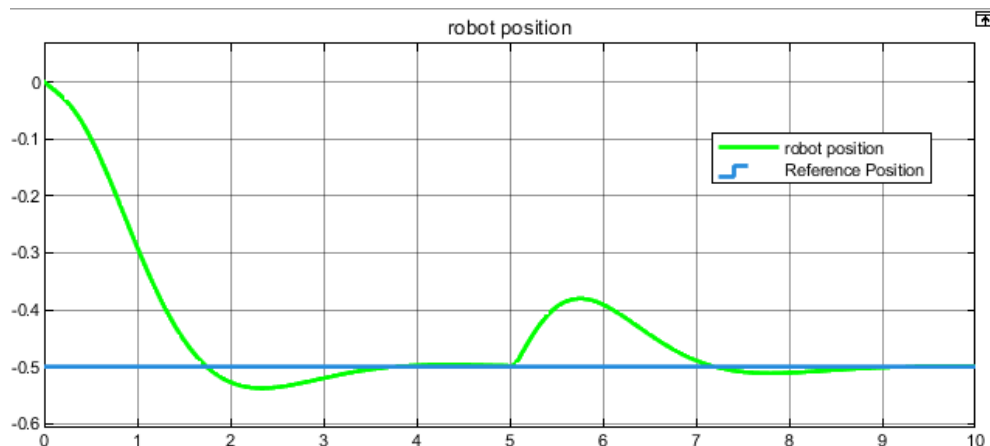


Figure 13: Position (x) v/s Time Graph from LQR Controlled Segway Robot Simulation in MATLAB-SIMULINK

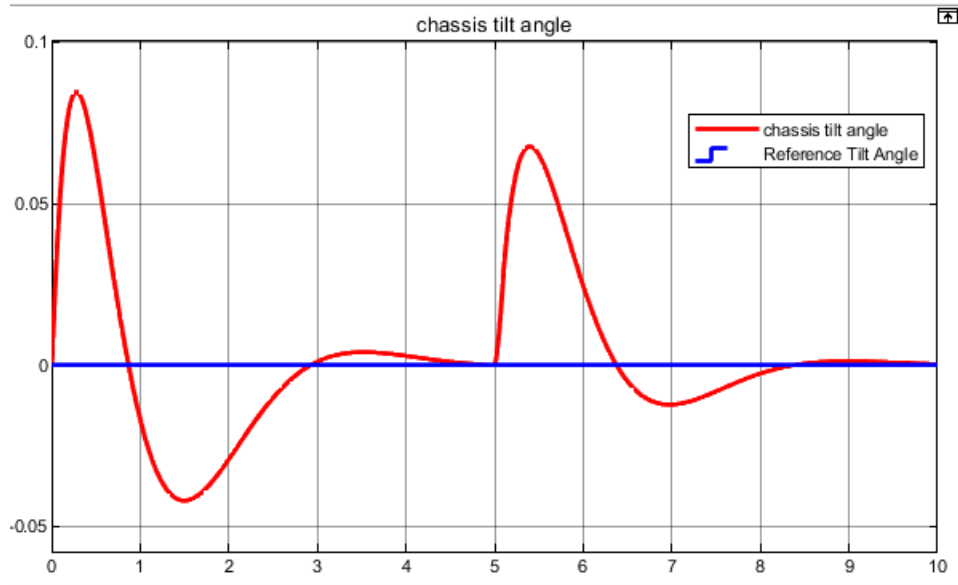


Figure 14: Chassis Tilt Angle (θ_c) v/s Time Graph from LQR Controlled Segway Robot Simulation in MATLAB-SIMULINK

Nevertheless, the manually tuned LQR still displays a compromise between fast position regulation and limited control effort. The selection of Q and R is based on engineering judgement, leaving room for further improvement via systematic optimization of the weights.

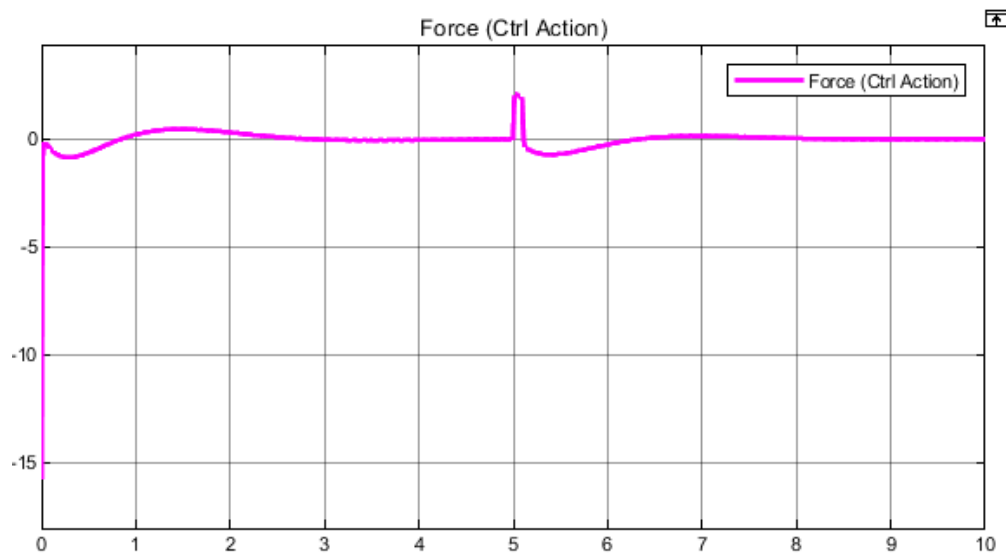


Figure 15: Control Action/Force (F_{input}) v/s Time Graph from LQR Controlled Segway Robot Simulation in MATLAB-SIMULINK

3.3 GA-Optimised LQR Results

The GA-LQR controller uses weights obtained from the GA procedure described in Section 2.3. The resulting responses show visibly faster position convergence and similar tilt-angle performance compared with the manually tuned LQR.

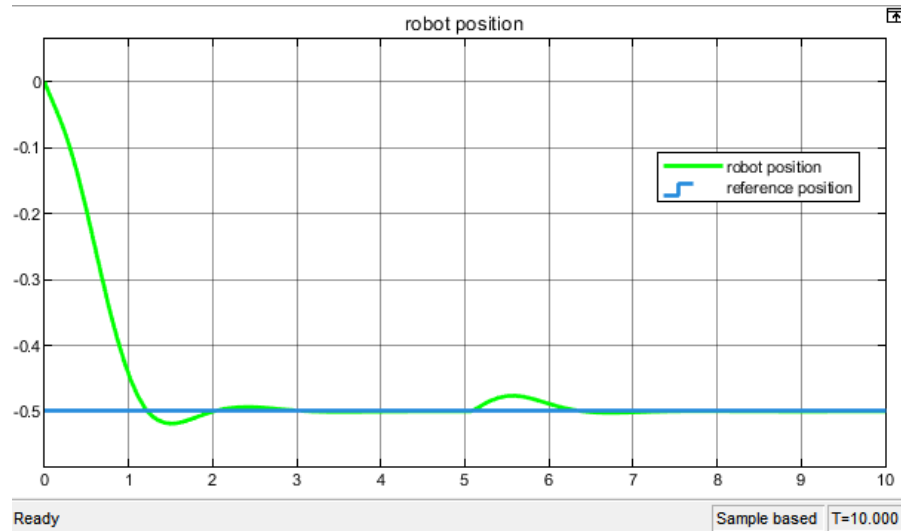


Figure 16: Position (x) v/s Time Graph from GA-Optimized LQR Controlled Segway Robot Simulation in MATLAB-SIMULINK

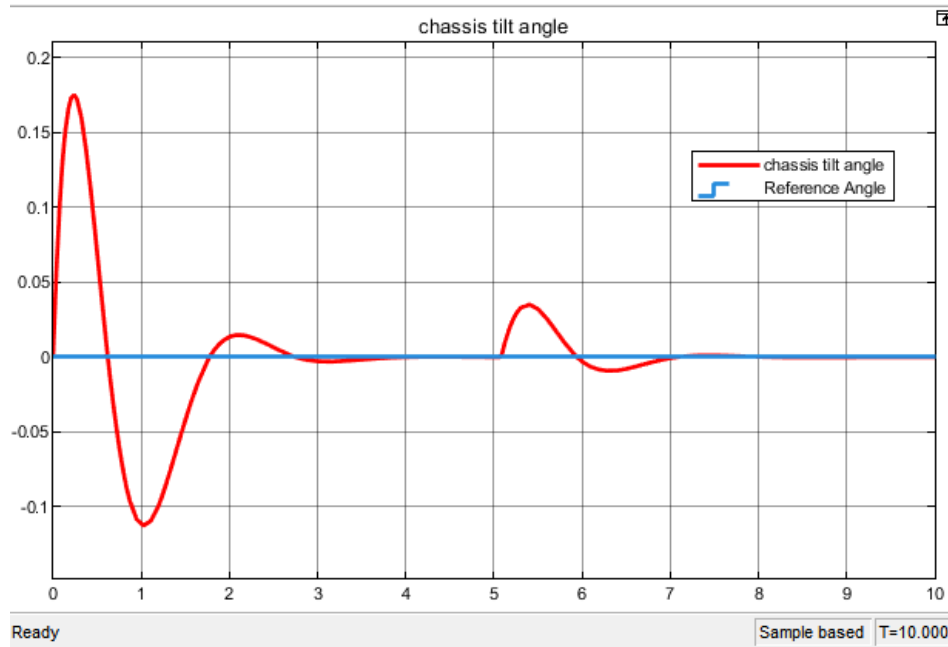


Figure 17: Chassis Tilt Angle (θ_c) v/s Time Graph from GA-Optimized LQR Controlled Segway Robot Simulation in MATLAB-SIMULINK

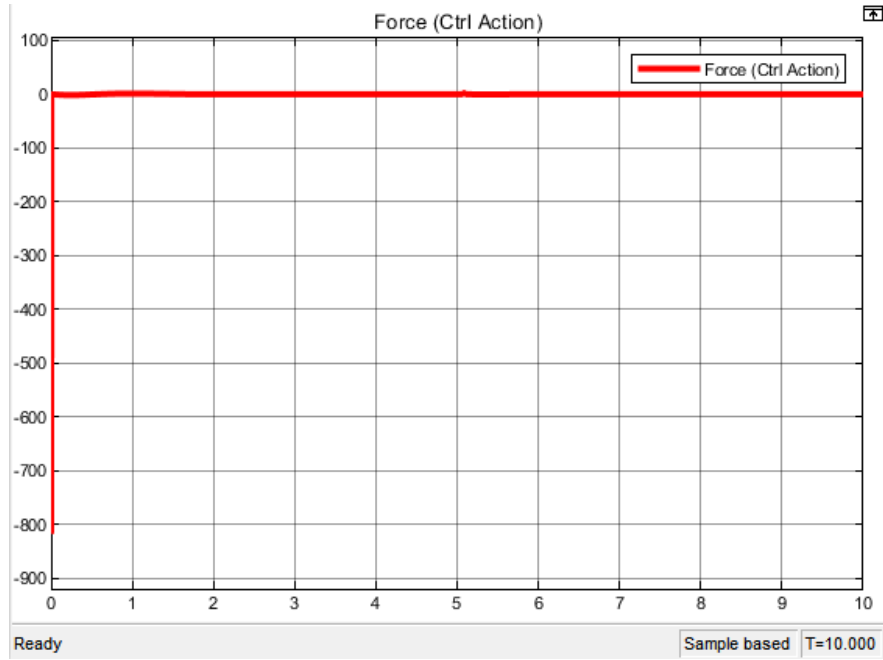


Figure 18: Control Action/Force (F_{input}) v/s Time Graph from GA-Optimized LQR Controlled Segway Robot Simulation in MATLAB-SIMULINK

The evolution of the GA fitness across generations confirms that the optimization converges: the ITAE-based cost decreases steadily and saturates by around the 40th generation, where the final solution is selected.

Pareto-optimal solutions and their costs:

q1	q2	q3	q4	r	K	Cost1	Cost2
15404	81159	26223	378.35	5.7676	{[-51.6801 -133.1505 356.0249 46.0283]}	2.7689	0.060158
19117	77314	31446	381.05	3.5873	{[-73.0006 -167.3424 442.5522 57.2329]}	2.2207	0.062709
86399	64007	41569	2709	1.5064	{[-239.4876 -280.1027 746.5198 104.4149]}	0.52434	0.075555
6.1386e+05	55017	20116	1070.8	0.22968	{[-1.6348e+03 -955.9911 2.0325e+03 274.0487]}	0.13904	0.1293
22952	77264	31908	942.59	3.4576	{[-81.4754 -172.9184 464.0277 61.4007]}	1.9044	0.064507
19117	77315	31447	381.11	3.7123	{[-71.7612 -164.5068 435.3261 56.2828]}	2.2209	0.062705
1.1151e+05	61563	28555	1186.6	0.5762	{[-439.9252 -454.4467 1.1210e+03 151.5097]}	0.40394	0.077226
6.1386e+05	55017	20116	1070.8	0.22968	{[-1.6348e+03 -955.9911 2.0325e+03 274.0487]}	0.13904	0.1293
46113	68249	42814	4959.9	3.315	{[-117.9418 -182.0203 531.3851 78.0275]}	0.95788	0.071646
31927	71548	34139	4920.7	3.1847	{[-100.1244 -182.2789 536.5411 79.1900]}	1.3705	0.068902
36516	68536	41990	3552.4	3.8479	{[-97.4154 -164.3020 472.0566 67.2377]}	1.1669	0.069833

Figure 19: Final Output of Genetic Algorithm After 40 Generations

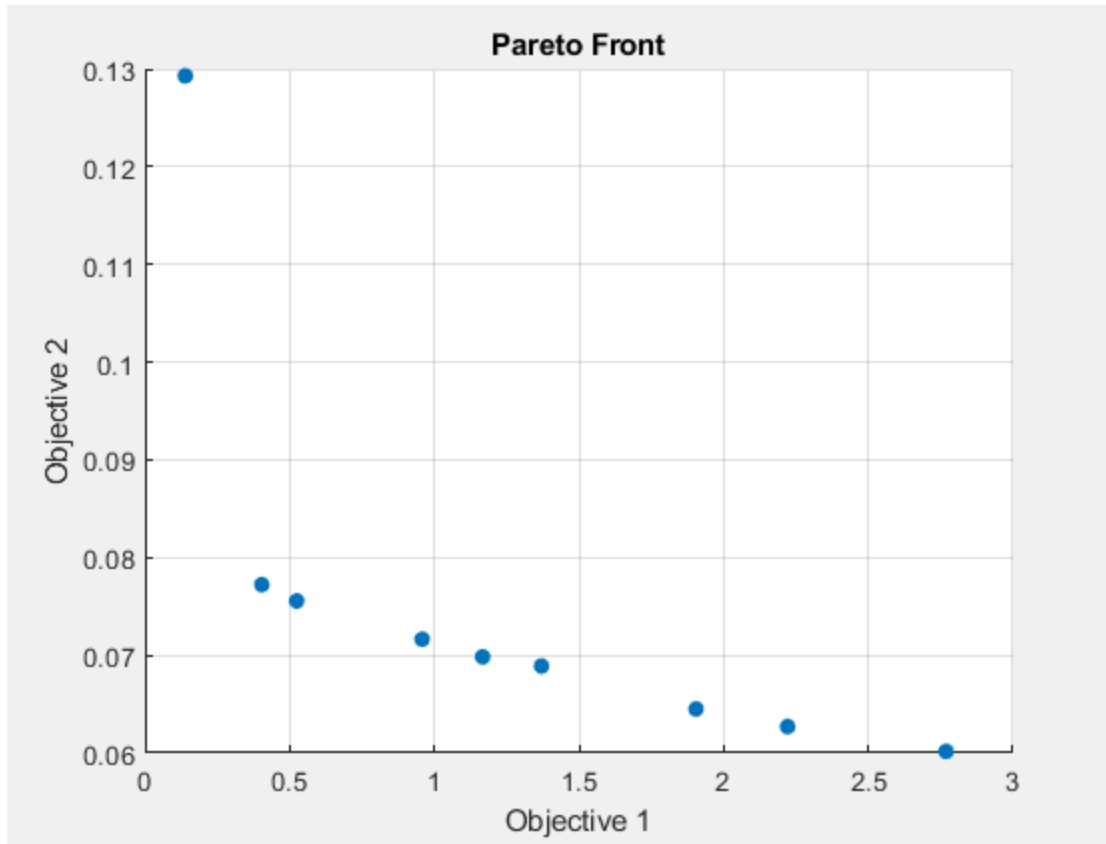


Figure 20: Pareto Front obtained from the GA run

Above figure shows the Pareto Front obtained from the GA run, where Objective 1 and Objective 2 correspond to the ITAE of cart-position error and chassis-tilt error respectively. The curve illustrates the trade-off between improving position regulation and maintaining small tilt-angle error; the final GA solution is selected from the lower-right part of this front, where both objectives are simultaneously small.

3.4 Quantitative Comparison of LQR and GA-LQR

To quantify the improvement due to GA tuning, the key time-domain performance metrics for the LQR and GA-LQR controllers are summarized in Table 3.1. The rise time and settling time are reported for both cart position and chassis tilt angle.

Controller	System Output	Rise Time (seconds)	Settling Time (seconds)
LQR – Manually Tuned	Position	1.16	8.17
	Chassis Tilt Angle	0.0014	9.99
LQR – GA Tuned	Position	0.85	6.03
	Chassis Tilt Angle	0.0004	9.98

Table 3 – Performance of Segway robot with LQR and GA-LQR controllers

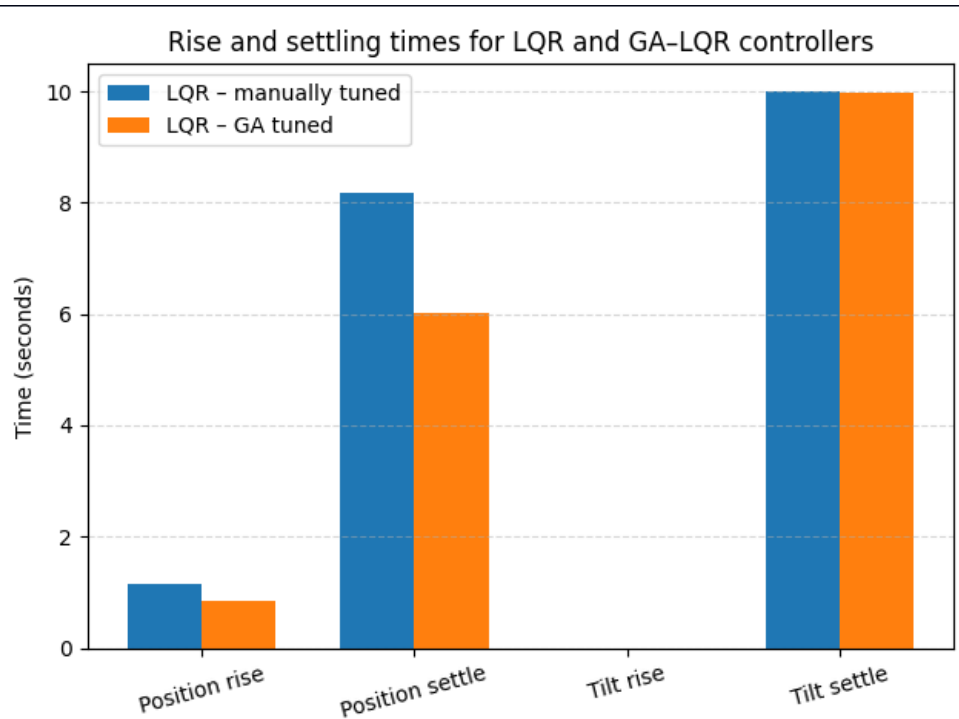


Figure 21: Comparison of rise and settling times for cart position and chassis tilt angle under manually tuned LQR and GA-tuned LQR controllers

The results show that GA tuning reduces the position rise time from 1.16 s to 0.85 s and the position settling time from 8.17 s to 6.03 s, while maintaining essentially identical tilt-angle settling time. This indicates that the GA-LQR controller

achieves a faster translational response without sacrificing tilt stability. The improvement is consistent with the ITAE-minimization objective used in the GA design, which penalizes late-time errors strongly.

CHAPTER 4

CONCLUSION

Across all controllers, the PID design provides a useful baseline that confirms stability of the torque-based plant but exhibits relatively slow and oscillatory behaviour. The manually tuned LQR, operating on the full state, already improves tilt damping and position tracking. The GA-optimised LQR further sharpens the transient response by systematically searching the Q – R space rather than relying on heuristic tuning, leading to the best overall performance among the three controllers.

These findings support the main research objective of demonstrating how an evolutionary optimisation layer can enhance an otherwise conventional LQR controller for a Segway-type self-balancing robot.

REFERENCES

1. V. Mudeng, B. Hassanah, Y. T. K. Priyanto, and O. Saputra, "Design and Simulation of Two-Wheeled Balancing Mobile Robot with PID Controller," *Int. J. Sustain. Transp. Technol.*, vol. 3, no. 1, pp. 12–19, Apr. 2020.
2. L. G. B. Putra, F. Wahab, and T. A. Tamba, "Design and Implementation of Linear Quadratic Regulator Control for Two-Wheeled Self-Balancing Robot," *Bull. Electr. Eng. Inform.*, vol. 14, no. 2, pp. 931–939, Apr. 2025.
3. I. K. Mohammed and A. I. Abdulla, "Balancing a Segway Robot Using LQR Controller Based on Genetic and Bacteria Foraging Optimization Algorithms," *TELKOMNIKA Telecommun. Comput. Electron. Control*, vol. 18, no. 5, pp. 2642–2653, Oct. 2020.
4. Feriyonika and A. Hidayat, "Balancing Control of Two-Wheeled Robot by Using Linear Quadratic Gaussian (LQG)," *J. Telecommun. Electron. Comput. Eng.*, vol. 12, no. 3, pp. 55–59, Sep. 2020.
5. O. A. Choudhry, M. Wasim, A. Ali, M. A. Choudhry, and J. Iqbal, "Modelling and Robust Controller Design for an Underactuated Self-Balancing Robot with Uncertain Parameter Estimation," *PLoS One*, vol. 18, no. 8, Aug. 2023, Art. no. e0285495.
6. B. R. Burkan and Ö. C. Özgüney, "Balance Control of Segway Robots Using Adaptive-Robust Controller," *Ind. 4.0, Int. Sci. J.*, vol. 2, no. 1, pp. 38–41, 2017.
7. Y. Su, T. Wang, K. Zhang, C. Yao, and Z. Wang, "Adaptive Nonlinear Control Algorithm for a Self-Balancing Robot," *IEEE Access*, vol. 8, pp. 3751–3760, 2020.
8. J. Chen, N. He, Z. Xu, M. Dou, and L. He, "Design and Implementation of a Novel Two-Wheeled Composite Self-Balancing Robot for Stationary Operations in Unknown Terrain," *IEEE Access*, vol. 13, pp. 86032–86045, May 2025.